



TopicLens AI

Tweet Topic Classification Experiments Using an NLP Pipeline

Student Name: Htut Myat Oo

Student ID: 239638957

Module: Machine Learning And Data Analytics (CETM47)

May 2026



Project Context, Objectives, and Viva Framing



Project Objective

Design and deploy a multi-class tweet topic classification pipeline for social media text analysis.

Industry-Aligned Workflow

Project framed using CRISP-DM methodology aligning technical work with business goals and viva presentation needs.

Scientific Experimentation

Iterative experiments from baseline to refined models demonstrate hypothesis testing and controlled evaluation.

Real-world Constraints

Consideration of noisy text, short documents, class imbalance, and semantic overlap shaped design and metrics.

CRISP-DM Workflow and Iterative Development Journey

CRISP-DM Phases Overview

CRISP-DM structures the project into six iterative phases: Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation, and Deployment.

Iterative Nature with Feedback Loops

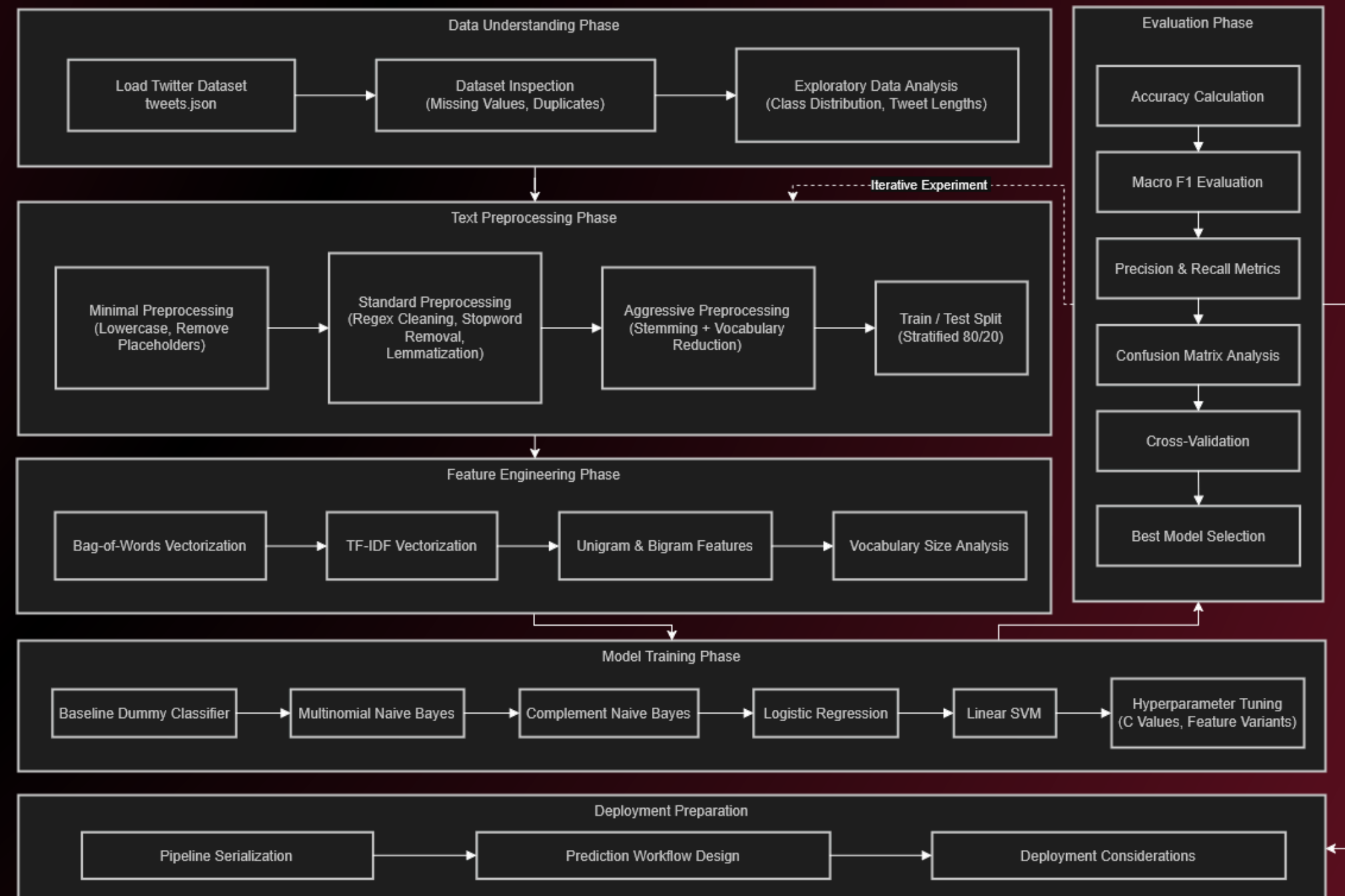
The methodology embraces feedback loops, especially from Evaluation back to Data Preparation and Modelling, ensuring continuous improvement.

Data Preparation and Modelling Strategy

Data preprocessing strategies and an experimental ladder of models optimize performance on imbalanced tweet datasets.

Evaluation and Deployment Focus

Evaluation uses stratified splits and Macro F1-score, guiding model tuning, while deployment ensures consistent production readiness.



Problem Definition and Business Understanding



Automated Tweet Classification

TopicLens AI classifies tweets into six key topical categories relevant to social media analysis and marketing.

Business Use Cases

Classification supports content moderation, targeted ads, trend detection, and personalized recommendations at scale.

Handling Class Imbalance

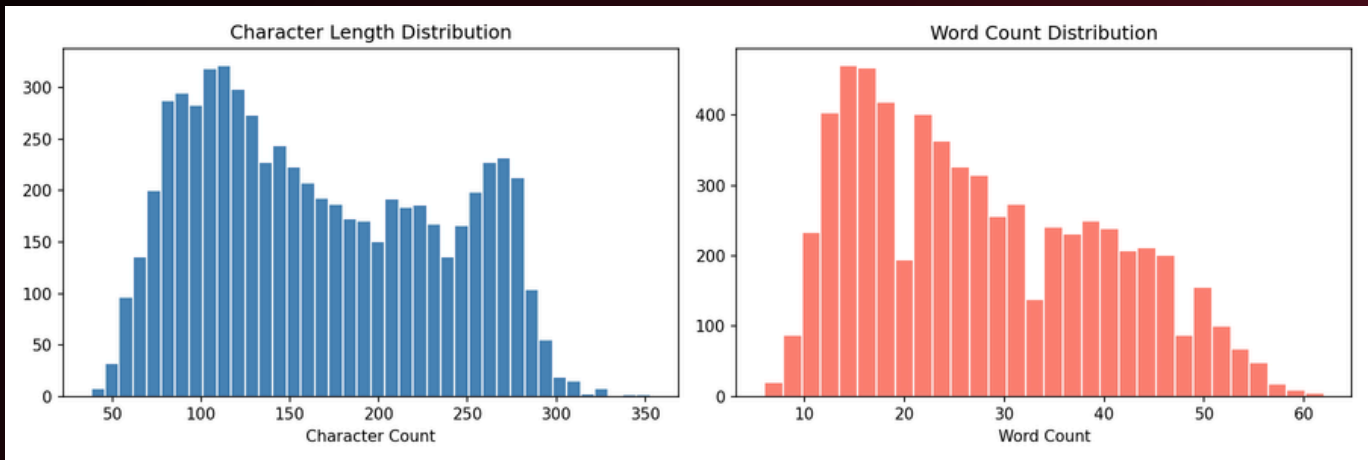
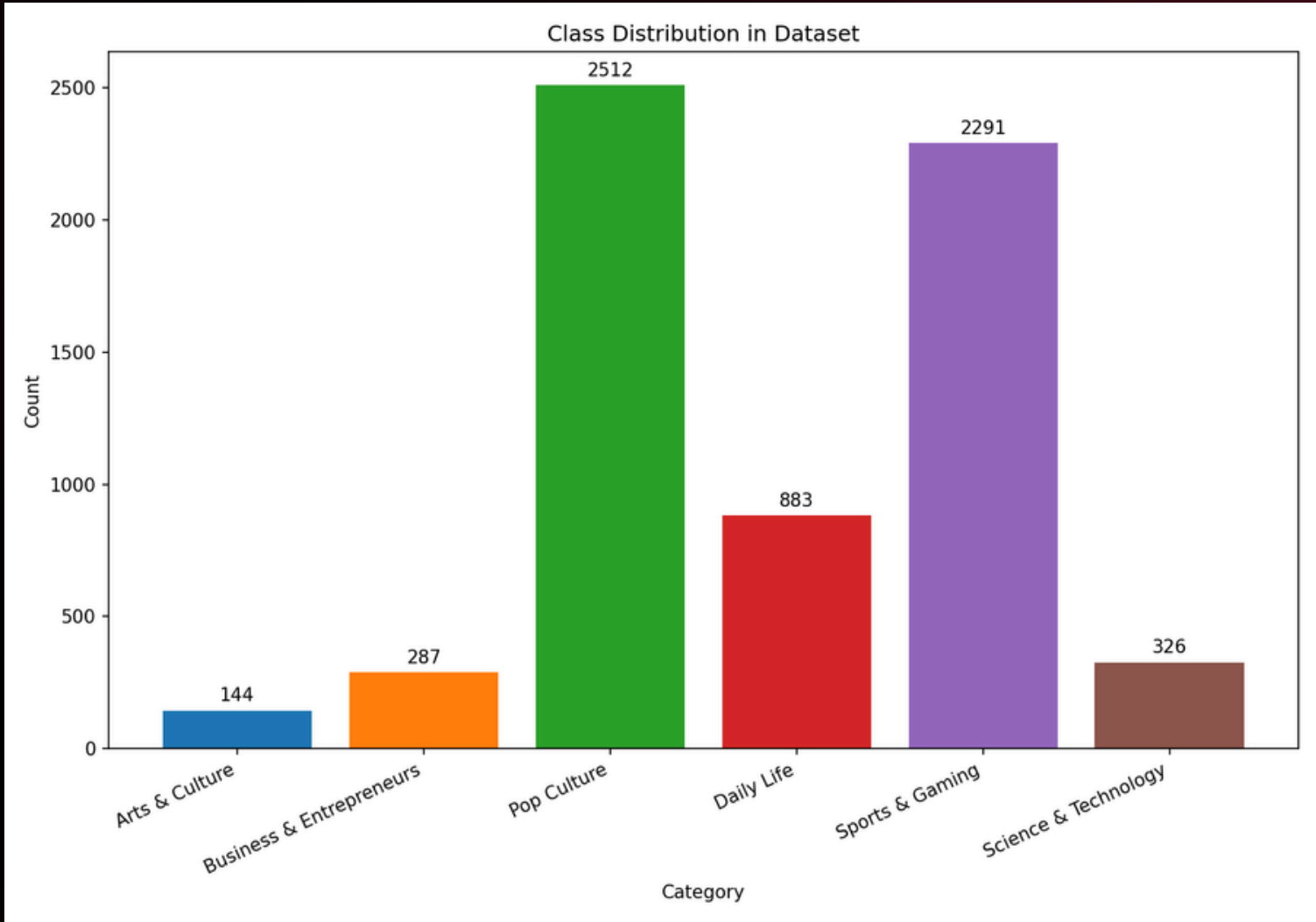
Robustness to imbalanced data ensured by prioritizing Macro F1-score to fairly evaluate all topic categories.

Inference Efficiency

Model is designed for lightweight, real-time prediction, focusing on classical ML over complex architectures.

Data Understanding and Exploratory Insights

Category	Approx. Proportion	Implication
Pop Culture	High	Dominant class; risks biasing accuracy
Sports & Gaming	High	Strong vocabulary signals
Daily Life	Moderate	Moderate overlap with other topics
Science & Technology	Moderate	Distinct terminology but fewer samples
Business & Entrepreneurs	Low	Mixed language patterns
Arts & Culture	Low	Minority class; evaluation sensitive



Data Understanding and Exploratory Insights (Cont.)

```
1 DATA_URL = "https://raw.githubusercontent.com/htutmyatoo/tweet-topic-classification-experiments/refs/heads/main/data/tweets.json"
2
3 response = requests.get(DATA_URL)
4 response.raise_for_status() # Raise error if download fails
5 data = response.json()
6
7 df = pd.DataFrame(data)
8 print(f"Dataset loaded: {len(df)} rows, {df.shape[1]} columns")
9 df.head()
```

Dataset loaded: 6443 rows, 5 columns

	text	date	label	id	label_name
0	The {@Clinton LumberKings@} beat the {@Cedar R...	2019-09-08	4	1170516324419866624	sports_&_gaming
1	I would rather hear Eli Gold announce this Aub...	2019-09-08	4	1170516440690176006	sports_&_gaming
2	Someone take my phone away, I'm trying to not ...	2019-09-08	4	1170516543387709440	sports_&_gaming
3	A year ago, Louisville struggled to beat an FC...	2019-09-08	4	1170516620466429953	sports_&_gaming
4	Anyone know why the #Dodgers #Orioles game nex...	2019-09-08	4	1170516711411310592	sports_&_gaming

```
1 # Check missing values
2 print("\n=== Missing Values ===")
3 print(df.isnull().sum())
```

```
=== Missing Values ===
text      0
date      0
label     0
id        0
label_name 0
text_length 0
word_count 0
dtype: int64
```

```
1 # Check duplicate rows
2 print("\n=== Duplicate Rows ===")
3 print(f"Duplicates: {df.duplicated().sum()}")
```

```
=== Duplicate Rows ===
Duplicates: 0
```

```
1 # Check data types
2 print("=== Column Data Types ===")
3 print(df.dtypes)
```

```
=== Column Data Types ===
text      object
date      object
label     int64
id        int64
label_name object
text_length int64
word_count int64
dtype: object
```


Data Preparation and Preprocessing Evolution

Preprocessing Level	Techniques Applied	Vocabulary Size
Raw	None	33,022
Minimal	Lowercasing, placeholder removal	24,834
Standard	Regex cleaning, stopwords, lemmatisation	14,730
Aggressive	Regex cleaning, stopwords, stemming	12,778

Original:

Three unanswered goals by {{USERNAME}} has tied the game for {{USERNAME}} 9-9. #MannCup

Minimal:

three unanswered goals by has tied the game for 9-9. #manncup

Standard:

three unanswered goal tied game manncup

Aggressive:

three unansw goal tie game manncup

Data Preparation and Preprocessing Evolution (Cont.)

```
1 stemmer = PorterStemmer()
2 lemmatizer = WordNetLemmatizer()
3 STOP_WORDS = set(stopwords.words('english'))
4
5 def preprocess_minimal(text):
6     """Preprocessing Level 1 - Minimal: lowercase + remove placeholder tokens only."""
7     text = text.lower()
8     # Remove {{URL}}, {{USERNAME}}, {@handle@} placeholders
9     text = re.sub(r'\{\{[^\}]+\}\}', '', text)
10    text = re.sub(r'\{@[^\}]+\}', '', text)
11    return text.strip()
12
13 def preprocess_standard(text):
14     """Preprocessing Level 2 - Standard: Level 1 + regex cleaning + stopword removal + lemmatisation."""
15    text = preprocess_minimal(text)
16    text = re.sub(r'http\S+|www.\S+', '', text) # Residual URLs
17    text = re.sub(r'#(\w+)', r'\1', text) # Keep hashtag text, remove '#'
18    text = re.sub(r'[^\w\s]', '', text) # Remove punctuation/numbers
19    text = re.sub(r'\s+', ' ', text).strip() # Normalise whitespace
20    tokens = word_tokenize(text)
21    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in STOP_WORDS and len(t) > 1]
22    return ' '.join(tokens)
23
24
25 def preprocess_aggressive(text):
26     """Preprocessing Level 3 - Aggressive: Level 2 but with stemming instead of lemmatisation."""
27    text = preprocess_minimal(text)
28    text = re.sub(r'http\S+|www.\S+', '', text)
29    text = re.sub(r'#(\w+)', r'\1', text)
30    text = re.sub(r'[^\w\s]', '', text)
31    text = re.sub(r'\s+', ' ', text).strip()
32    tokens = word_tokenize(text)
33    tokens = [stemmer.stem(t) for t in tokens if t not in STOP_WORDS and len(t) > 1]
34    return ' '.join(tokens)
35
```


Modelling Strategy and NLP Pipeline Design

Baseline Model Evaluation

Initial dummy classifier highlighted dataset imbalance and the gap between accuracy and Macro F1-score metrics.

Feature Representation Enhancements

Progression from Bag-of-Words to TF-IDF and from unigrams to bigrams improved semantic capture effectively.

Classification Models Compared

Logistic Regression provided a strong baseline, but Linear SVM achieved best performance through margin maximisation.

Pipeline Design and Optimization

A reusable NLP pipeline combined preprocessing, vectorisation, classification, and hyperparameter tuning for fair evaluation.

Experiment Design

I used the same stratified 80/20 train-test split in all experiments for fair comparison.

Exp	Preprocessing	Vectorisation	Classifier
E0	None	—	Dummy (majority) — Baseline
E1	Minimal	BoW (unigram)	Naïve Bayes (Multinomial)
E2	Standard	BoW (unigram)	Naïve Bayes (Multinomial)
E3	Standard	TF-IDF (unigram)	Naïve Bayes (Complement)
E4	Standard	TF-IDF (1+2-gram)	Logistic Regression
E5	Aggressive	TF-IDF (1+2-gram)	Logistic Regression
E6	Standard	TF-IDF (1+2-gram)	Linear SVM
E7	Standard	TF-IDF (1+2-gram)	Linear SVM + tuned C

Modelling Strategy and NLP Pipeline Design (Cont.)

4.2.1 E0: Baseline – Majority-class Dummy classifier

```
1 dummy = DummyClassifier(strategy='most_frequent', random_state=RANDOM_STATE)
2 dummy.fit(X_train_min, y_train)
3 y_pred_e0 = dummy.predict(X_test_min)
4 acc_e0, f1_e0 = evaluate('E0 - Baseline (Dummy)', y_test, y_pred_e0)
5 print(f"E0 Baseline \nAccuracy: {acc_e0:.4f} | Macro F1: {f1_e0:.4f}")
```

E0 Baseline
Accuracy: 0.3902 | Macro F1: 0.0936

4.2.2 E1: BoW (Minimal preprocessing) + Multinomial Naïve Bayes

```
1 bow_min = CountVectorizer()
2 X_tr_e1 = bow_min.fit_transform(X_train_min)
3 X_te_e1 = bow_min.transform(X_test_min)
4
5 mnb_e1 = MultinomialNB()
6 mnb_e1.fit(X_tr_e1, y_train)
7 y_pred_e1 = mnb_e1.predict(X_te_e1)
8 acc_e1, f1_e1 = evaluate('E1 - BoW (Minimal) + MNB', y_test, y_pred_e1)
9 print(f"E1 BoW (Minimal preprocessing) + Multinomial Naïve Bayes \nAccuracy: {acc_e1:.4f} | Macro F1: {f1_e1:.4f}")
```

E1 BoW (Minimal preprocessing) + Multinomial Naïve Bayes
Accuracy: 0.7595 | Macro F1: 0.4621

4.2.3 E2: BoW (Standard preprocessing) + Multinomial Naïve Bayes

```
1 bow_std = CountVectorizer()
2 X_tr_e2 = bow_std.fit_transform(X_train_std)
3 X_te_e2 = bow_std.transform(X_test_std)
4
5 mnb_e2 = MultinomialNB()
6 mnb_e2.fit(X_tr_e2, y_train)
7 y_pred_e2 = mnb_e2.predict(X_te_e2)
8 acc_e2, f1_e2 = evaluate('E2 - BoW (Standard) + MNB', y_test, y_pred_e2)
9 print(f"E2 BoW (Standard preprocessing) + Multinomial Naïve Bayes \nAccuracy: {acc_e2:.4f} | Macro F1: {f1_e2:.4f}")
```

E2 BoW (Standard preprocessing) + Multinomial Naïve Bayes
Accuracy: 0.7773 | Macro F1: 0.5291

4.2.4 E3: TF-IDF (Standard) + Complement Naïve Bayes

```
1 tfidf_uni = TfidfVectorizer(ngram_range=(1, 1))
2 X_tr_e3 = tfidf_uni.fit_transform(X_train_std)
3 X_te_e3 = tfidf_uni.transform(X_test_std)
4
5 cnb_e3 = ComplementNB()
6 cnb_e3.fit(X_tr_e3, y_train)
7 y_pred_e3 = cnb_e3.predict(X_te_e3)
8 acc_e3, f1_e3 = evaluate('E3 - TF-IDF (Uni) + CNB', y_test, y_pred_e3)
9 print(f"E3 TF-IDF (Standard) + Complement Naïve Bayes \nAccuracy: {acc_e3:.4f} | Macro F1: {f1_e3:.4f}")
```

E3 TF-IDF (Standard) + Complement Naïve Bayes
Accuracy: 0.7952 | Macro F1: 0.6242

Modelling Strategy and NLP Pipeline Design (Cont.)

4.2.5 E4: TF-IDF Bigram (Standard) + Logistic Regression

```
1 tfidf_bi = TfidfVectorizer(ngram_range=(1, 2), max_features=50000)
2 X_tr_e4 = tfidf_bi.fit_transform(X_train_std)
3 X_te_e4 = tfidf_bi.transform(X_test_std)
4
5 lr_e4 = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
6 lr_e4.fit(X_tr_e4, y_train)
7 y_pred_e4 = lr_e4.predict(X_te_e4)
8 acc_e4, f1_e4 = evaluate('E4 - TF-IDF Bigram + LR', y_test, y_pred_e4)
9 print(f"E4 TF-IDF Bigram (Standard) + Logistic Regression \nAccuracy: {acc_e4:.4f} | Macro F1: {f1_e4:.4f}")
```

E4 TF-IDF Bigram (Standard) + Logistic Regression
Accuracy: 0.7595 | Macro F1: 0.4770

4.2.6 E5: TF-IDF Bigram (Aggressive) + Logistic Regression

```
1 tfidf_bi_agg = TfidfVectorizer(ngram_range=(1, 2), max_features=50000)
2 X_tr_e5 = tfidf_bi_agg.fit_transform(X_train_agg)
3 X_te_e5 = tfidf_bi_agg.transform(X_test_agg)
4
5 lr_e5 = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
6 lr_e5.fit(X_tr_e5, y_train)
7 y_pred_e5 = lr_e5.predict(X_te_e5)
8 acc_e5, f1_e5 = evaluate('E5 - TF-IDF Bigram (Agg.) + LR', y_test, y_pred_e5)
9 print(f"E5 TF-IDF Bigram (Aggressive) + Logistic Regression \nAccuracy: {acc_e5:.4f} | Macro F1: {f1_e5:.4f}")
```

E5 TF-IDF Bigram (Aggressive) + Logistic Regression
Accuracy: 0.7595 | Macro F1: 0.4739

4.2.7 E6: TF-IDF Bigram (Standard) + Linear SVM (default C)

```
1 svm_e6 = LinearSVC(max_iter=2000, random_state=RANDOM_STATE)
2 svm_e6.fit(X_tr_e4, y_train) # Reuse E4 feature matrix
3 y_pred_e6 = svm_e6.predict(X_te_e4)
4 acc_e6, f1_e6 = evaluate('E6 - TF-IDF Bigram + LinearSVM (C=1)', y_test, y_pred_e6)
5 print(f"E6 TF-IDF Bigram (Standard) + Linear SVM (default C) \nAccuracy: {acc_e6:.4f} | Macro F1: {f1_e6:.4f}")
```

E6 TF-IDF Bigram (Standard) + Linear SVM (default C)
Accuracy: 0.8061 | Macro F1: 0.6478

4.2.8 E7: TF-IDF Bigram (Standard) + Linear SVM (tuned C via CV)

```
1 # Data-driven hyperparameter selection using cross-validation
2 from sklearn.model_selection import GridSearchCV
3
4 param_grid = {'C': [0.01, 0.1, 1.0, 5.0, 10.0]}
5 grid_search = GridSearchCV(
6     LinearSVC(max_iter=3000, random_state=RANDOM_STATE),
7     param_grid,
8     cv=5,
9     scoring='f1_macro',
10    n_jobs=-1
11 )
12 grid_search.fit(X_tr_e4, y_train)
13 best_C = grid_search.best_params_['C']
14 print(f"Best C found: {best_C} (CV Macro F1: {grid_search.best_score_:.4f})")
```

Best C found: 10.0 (CV Macro F1: 0.5904)

```
1 y_pred_e7 = grid_search.best_estimator_.predict(X_te_e4)
2 acc_e7, f1_e7 = evaluate(f'E7 - TF-IDF Bigram + LinearSVM (C={best_C})', y_test, y_pred_e7)
3 print(f"E7 TF-IDF Bigram (Standard) + Linear SVM (tuned C via CV) \nAccuracy: {acc_e7:.4f} | Macro F1: {f1_e7:.4f}")
```

E7 TF-IDF Bigram (Standard) + Linear SVM (tuned C via CV)
Accuracy: 0.8115 | Macro F1: 0.6622

Experimental Results and Performance Progression

Baseline Model Evaluation

The Dummy Classifier showed accuracy of 39% but very low Macro F1, exposing accuracy's limitations on imbalanced data.

Bag-of-Words and Naïve Bayes Gains

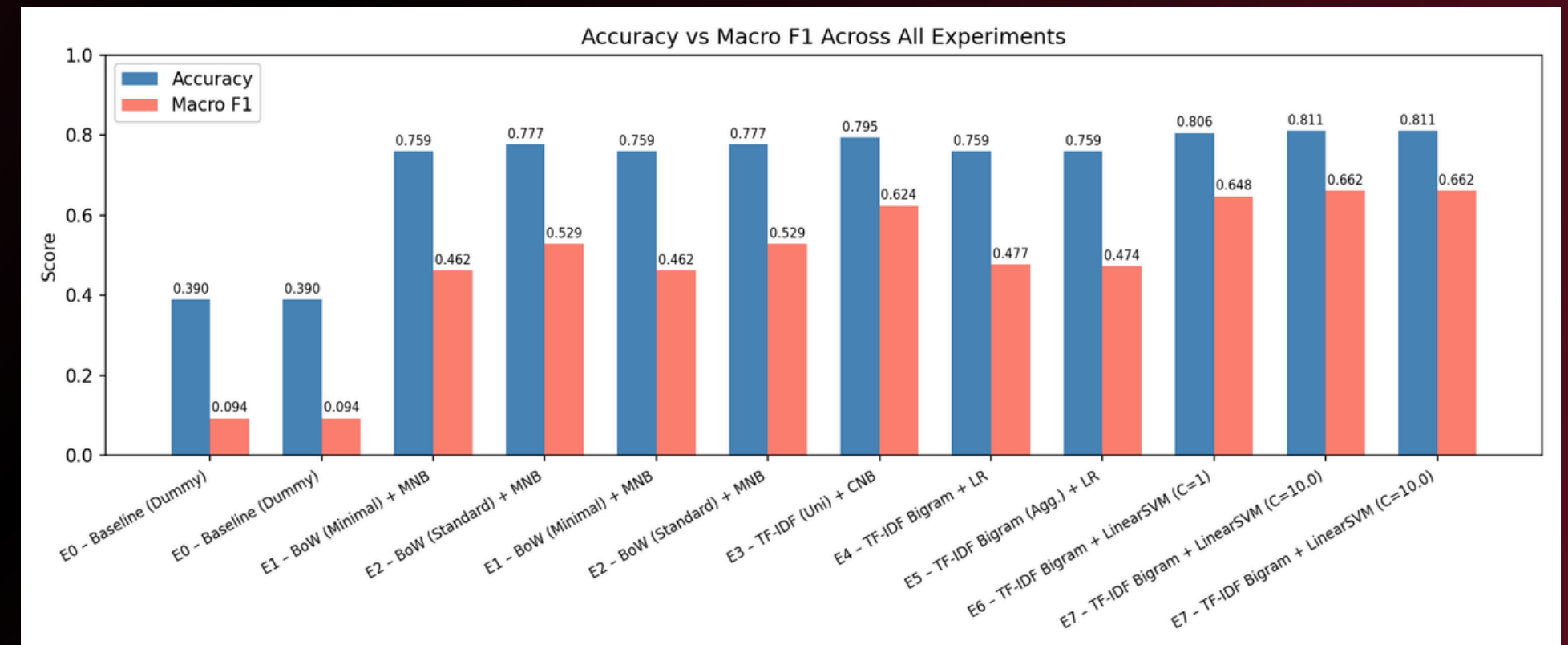
Using Bag-of-Words with Naïve Bayes increased Macro F1 to over 0.52, marking substantial improvement over baseline.

TF-IDF and Complement Naïve Bayes

TF-IDF representations with Complement Naïve Bayes further enhanced performance by reducing uninformative tokens.

Linear SVM Best Performance

Linear SVM achieved the highest accuracy (~81%) and Macro F1 (0.66), outperforming other models with balanced metrics.



Evaluation Metrics, Error Analysis, and Insights

Fair Evaluation Metric

Macro F1-score balances class importance, ensuring minority classes are fairly evaluated.

Confusion Matrix Insights

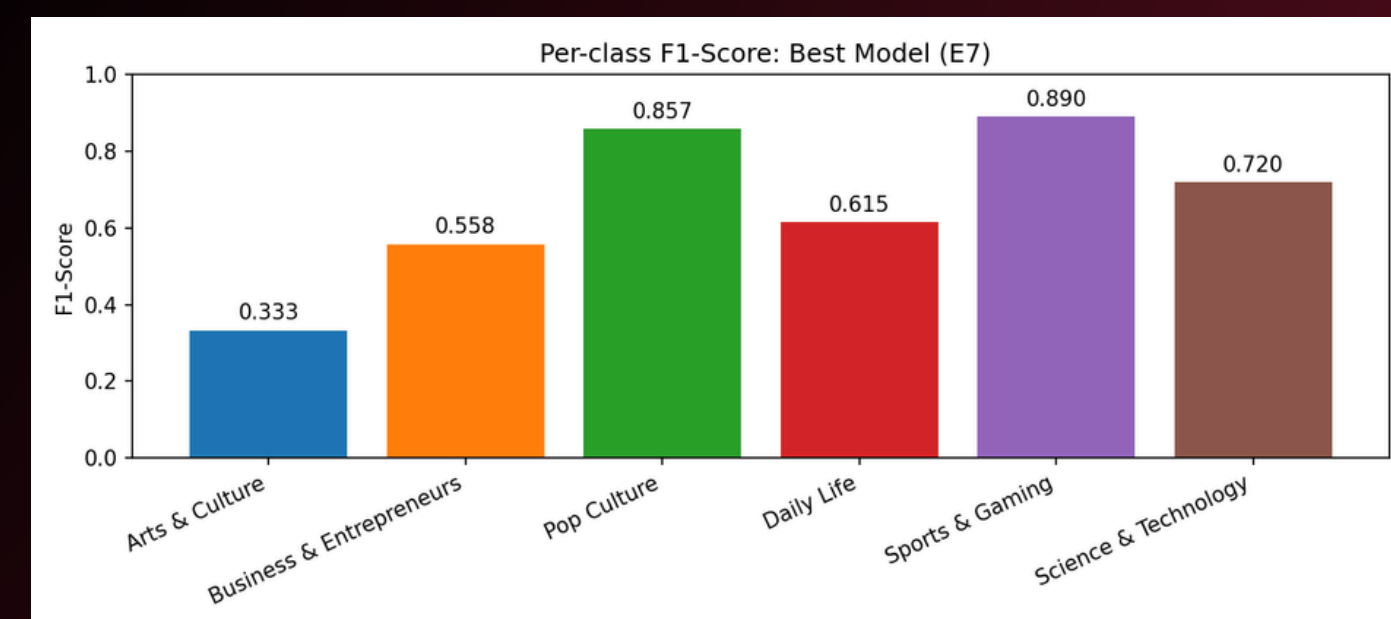
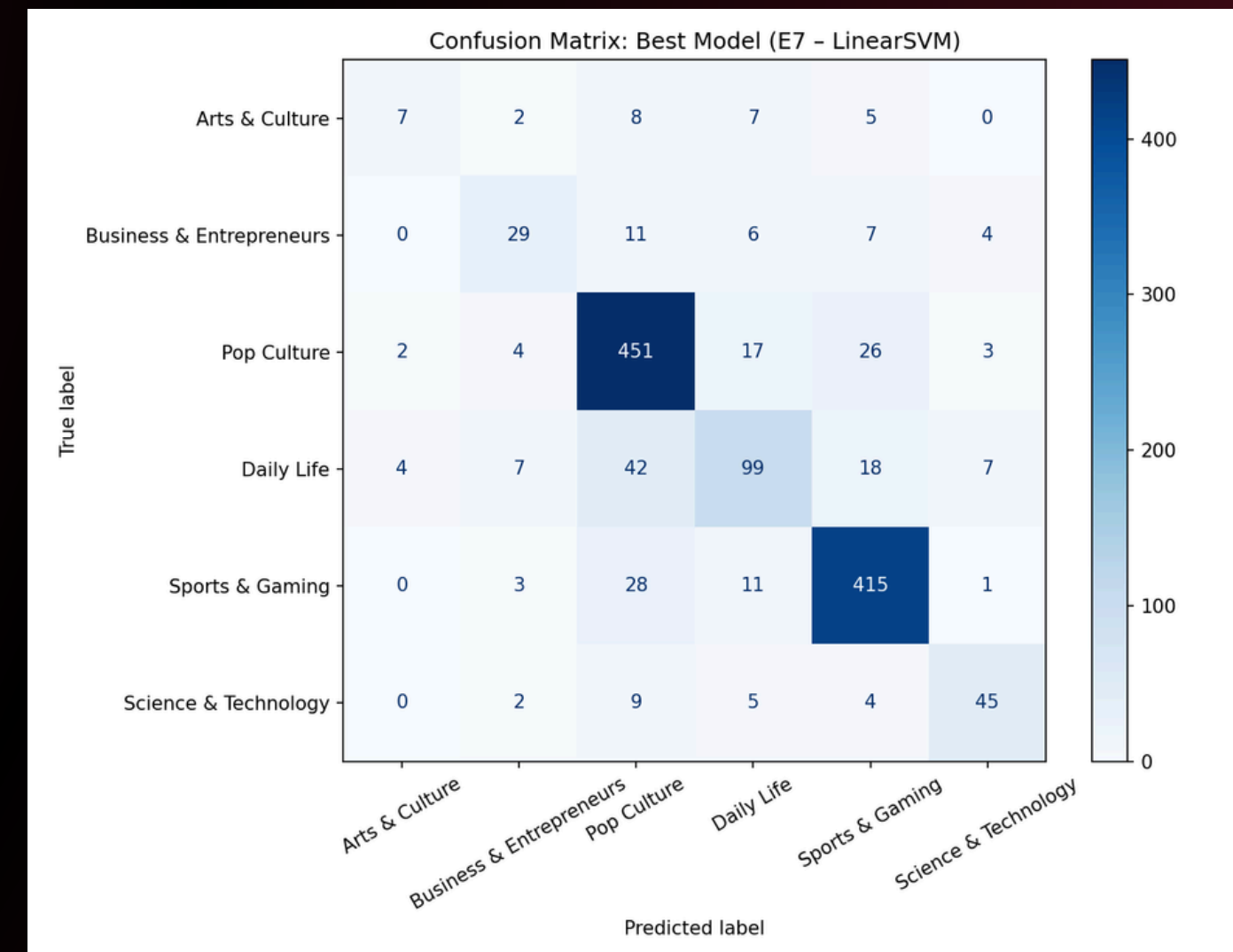
Strong diagonal dominance indicates distinct class vocabularies, while overlaps reveal semantic ambiguity.

Per-Class Performance Visualization

Bar charts illustrate disparities in F1 scores, making evaluation insights accessible.

Model Stability and Future Directions

Cross-validation shows low variance, guiding targeted improvements like data augmentation.



Evaluation Metrics, Error Analysis, and Insights (Cont.)

5.7 Cross-validation stability check for best model

```
1 # Assess reliability of best model using 5-fold stratified CV
2 pipeline_best = Pipeline([
3     ('tfidf', TfidfVectorizer(ngram_range=(1, 2), max_features=50000)),
4     ('clf', LinearSVC(C=best_C, max_iter=3000, random_state=RANDOM_STATE))
5 ])
6
7 X_std_all = pd.concat([X_train_std, X_test_std]) # Use full dataset for CV
8 y_all = pd.concat([y_train, y_test])
9
10 cv_scores = cross_val_score(pipeline_best, X_std_all, y_all,
11                             cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
12                             scoring='f1_macro', n_jobs=-1)
13
14 print(f"5-Fold CV Macro F1 scores: {cv_scores.round(4)}")
15 print(f"Mean: {cv_scores.mean():.4f} | Std: {cv_scores.std():.4f}")
```

```
... 5-Fold CV Macro F1 scores: [0.6815 0.6563 0.629 0.6615 0.6461]
Mean: 0.6549 | Std: 0.0173
```


Deployment, Inference Pipeline, and Production Considerations

Model Deployment and Serialization

The final Linear SVM pipeline with TF-IDF vectorization is serialized using joblib for consistent inference reuse.

Real-time Inference Interface

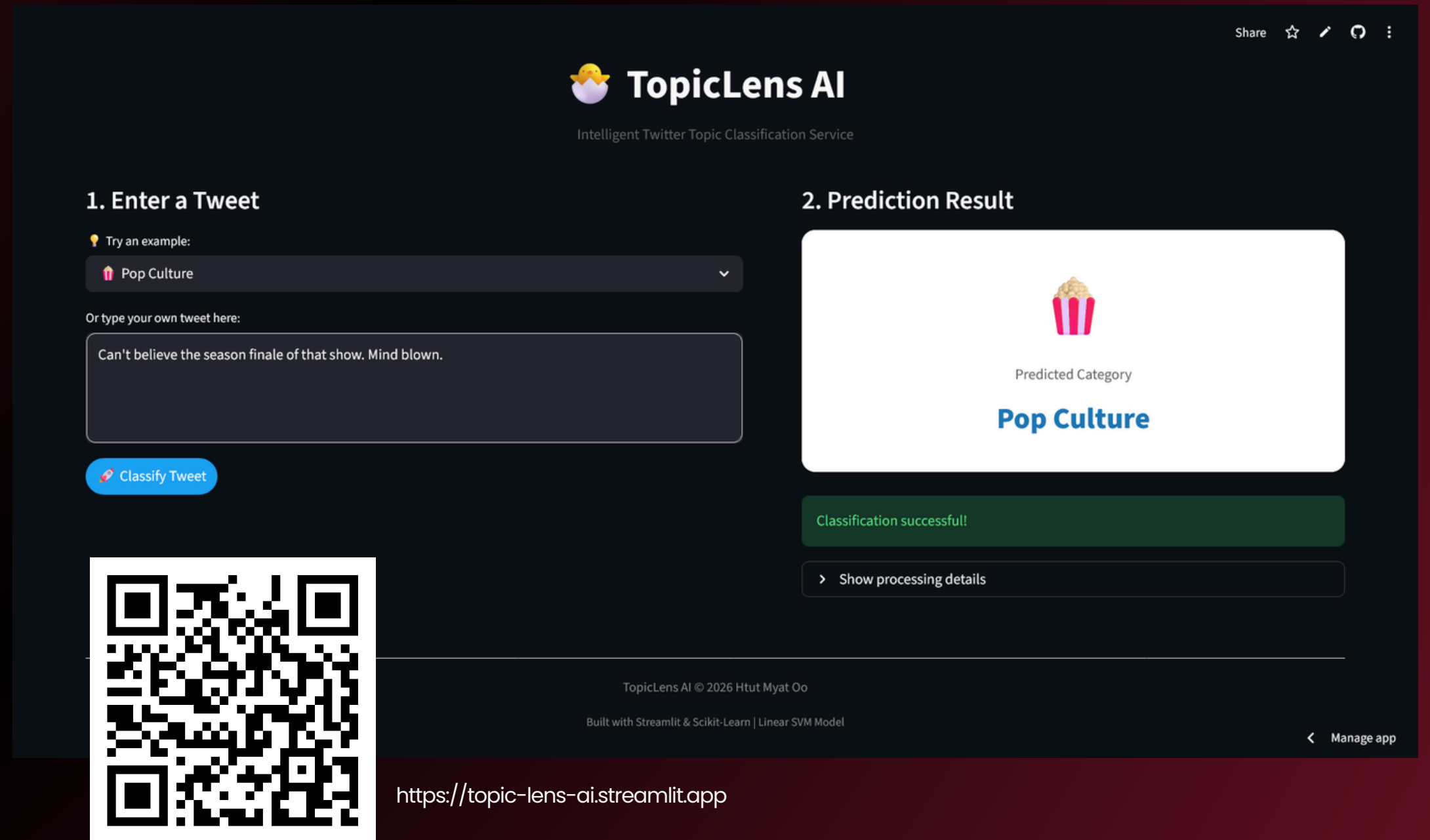
A Streamlit app enables real-time tweet classification via a user-friendly, lightweight interface.

Production System Considerations

REST API exposure facilitates horizontal scaling; monitoring, latency, model versioning, and data drift are key factors.

Ongoing Maintenance and Evaluation

Periodic retraining and evaluation ensure sustained model accuracy as language and topics evolve over time.



The screenshot displays the TopicLens AI web interface, which is a Streamlit application for tweet classification. The interface is dark-themed and features a header with the app's logo and title, "TopicLens AI", and a subtitle, "Intelligent Twitter Topic Classification Service". In the top right corner, there are icons for sharing, favoriting, editing, and a menu. The main content area is divided into two columns. The left column, titled "1. Enter a Tweet", contains a dropdown menu with "Pop Culture" selected, a text input field with the example tweet "Can't believe the season finale of that show. Mind blown.", and a blue "Classify Tweet" button. Below this is a large QR code. The right column, titled "2. Prediction Result", shows a white box with a popcorn icon and the text "Predicted Category" followed by "Pop Culture" in blue. Below this is a green "Classification successful!" message and a button to "Show processing details". At the bottom of the interface, there is a footer with the copyright notice "TopicLens AI © 2026 Htut Myat Oo", the text "Built with Streamlit & Scikit-Learn | Linear SVM Model", and a "Manage app" link. The URL "https://topic-lens-ai.streamlit.app" is displayed at the bottom of the slide.

TopicLens AI
Intelligent Twitter Topic Classification Service

1. Enter a Tweet

Try an example:
Pop Culture

Or type your own tweet here:
Can't believe the season finale of that show. Mind blown.

Classify Tweet

2. Prediction Result

Predicted Category
Pop Culture

Classification successful!

Show processing details

TopicLens AI © 2026 Htut Myat Oo
Built with Streamlit & Scikit-Learn | Linear SVM Model

Manage app

<https://topic-lens-ai.streamlit.app>

Deployment, Inference Pipeline, and Production Considerations (Cont.)

```
1 import joblib
2
3 joblib.dump(pipeline_final, path + 'model/best_tweet_classifier.pkl')
4 print("Model saved as best_tweet_classifier.pkl")
```

Model saved as best_tweet_classifier.pkl

saved best model from notebook

```
1 # =====
2 # 2. APP INITIALIZATION & CACHING
3 # =====
4 @st.cache_resource(show_spinner="Initializing NLP Engine...")
5 def initialize_app():
6     for resource in ['stopwords', 'punkt', 'wordnet', 'punkt_tab']:
7         nltk.download(resource, quiet=True)
8
9     try:
10         model = joblib.load('model/best_tweet_classifier.pkl') # model loading
11         return model
12     except FileNotFoundError:
13         st.error("Model file 'best_tweet_classifier.pkl' not found.")
14         return None
15
16 model = initialize_app()
17 lemmatizer = WordNetLemmatizer()
18 try:
19     STOP_WORDS = set(stopwords.words('english'))
20 except:
21     STOP_WORDS = set()
22
23 LABEL_MAP = {
24     0: ('Arts & Culture', '🎨'),
25     1: ('Business & Entrepreneurs', '💼'),
26     2: ('Pop Culture', '🎵'),
27     3: ('Daily Life', '👤'),
28     4: ('Sports & Gaming', '🎮'),
29     5: ('Science & Technology', '🚀')
30 }
31
32 # =====
33 # 3. PREPROCESSING PIPELINE
34 # =====
35 def preprocess_minimal(text):
36     text = text.lower()
37     text = re.sub(r'\{[^\}]+\}', '', text)
38     text = re.sub(r'\{@[^\}]+\}', '', text)
39     return text.strip()
40
41 def preprocess_standard(text):
42     text = preprocess_minimal(text)
43     text = re.sub(r'http[s+|www]\S+', '', text)
44     text = re.sub(r'#(\w+)', r'\1', text)
45     text = re.sub(r'^a-z\s', ' ', text)
46     text = re.sub(r'\s+', ' ', text).strip()
47     tokens = word_tokenize(text)
48     tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in STOP_WORDS and len(t) > 1]
49     return ' '.join(tokens)
```

load into streamlit app with same preprocessing pipeline

Conclusion, Reflection, and Future Directions

CRISP-DM Methodology Impact

A structured CRISP-DM approach effectively transformed social media text into a deployable NLP system.

Effective Model and Features

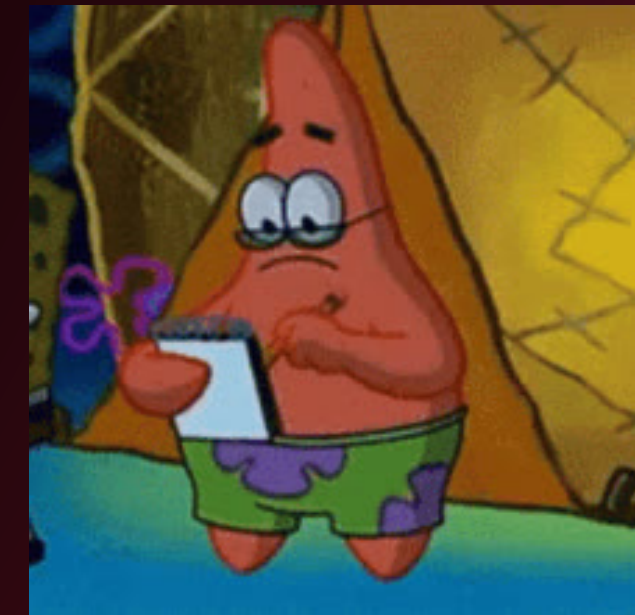
Linear SVM with TF-IDF bigrams balanced accuracy, interpretability, and efficiency in the NLP system.

Reflection and Iterative Improvement

Systematic experimentation and fair evaluation enabled iterative refinement and improved outcomes.

Future Directions in NLP

Future work includes transformer models, explainable AI, and live social media data integration.



Thankyou

